# Portfolio 2: queries and optimisation

Run the sales queries after the sample inserts, before the transaction exercise. All results below are expected values calculated from that data, not captured SQL Server output.

```
USE GreenBasketDB;
GO

-- 1. Products costing at most NGN 15,000.
SELECT ProductName, Price
FROM dbo.Products
WHERE Price <= 15000.00
ORDER BY Price;

-- 2. Customer name for each order.
SELECT o.OrderID, c.CustomerName, o.OrderDate
FROM dbo.Orders AS o
INNER JOIN dbo.Customers AS c
 ON o.CustomerID = c.CustomerID
ORDER BY o.OrderID;

-- 3. Total amount of each order.
SELECT OrderID, SUM(Quantity * UnitPrice) AS OrderTotal
FROM dbo.OrderItems
GROUP BY OrderID
ORDER BY OrderID;

-- 4. Total sales value by product.
SELECT p.ProductName,
       SUM(i.Quantity * i.UnitPrice) AS SalesAmount
FROM dbo.OrderItems AS i
INNER JOIN dbo.Products AS p ON i.ProductID = p.ProductID
GROUP BY p.ProductID, p.ProductName
ORDER BY SalesAmount DESC;

-- 5. Customers with no orders.
SELECT c.CustomerName
FROM dbo.Customers AS c
LEFT JOIN dbo.Orders AS o ON c.CustomerID = o.CustomerID
WHERE o.OrderID IS NULL;

-- 6. September orders; use a date range.
SELECT OrderID, OrderDate
FROM dbo.Orders
WHERE OrderDate >= '20260901' AND OrderDate < '20261001'
ORDER BY OrderID;
```

## Expected sales-query results

| Query | Expected result |
|---|---|
| 1 | Cooking oil: 8000.00; Beans bag: 15000.00. |
| 2 | Order 1: Ada; order 2: Bayo; order 3: Ada. |
| 3 | Order 1: 58000.00; order 2: 45000.00; order 3: |
| 4 | Rice bag: 50000.00; Beans bag: 45000.00; Cooking oil: |
| 5 | Chidi Obi. |
| 6 | Orders 1, 2 and 3. |

## Explain the SQL

WHERE selects matching rows. INNER JOIN connects matching rows in two tables. LEFT JOIN also keeps customers without a matching order. SUM adds amounts and GROUP BY calculates one sum per group. AS gives a short name to a table or result column. ORDER BY controls display order.

## A fair before-and-after index exercise

Three products are too few for a useful comparison. The small WHILE loop below adds 2,000 fictional practice products. @Number is a counter; SET increases it by one until it reaches 3000. CAST converts the number to text for the product name. Run this loader once; do not include these products in real sales.

In SSMS, select Query > Include Actual Execution Plan before executing the measurement. Save the plan with a descriptive .sqlplan filename. In Messages, record Products logical reads, CPU time and elapsed time. Logical reads count data pages accessed, not returned rows.

```
USE GreenBasketDB;
GO

-- Run once BEFORE creating the index in PDF 07.
-- Synthetic products are only for this performance exercise.
DECLARE @Number INT = 1000;
WHILE @Number < 3000
BEGIN
 INSERT INTO dbo.Products
        (ProductID, ProductName, Price, StockQuantity)
 VALUES
        (@Number,
         N'Practice product ' + CAST(@Number AS NVARCHAR(10)),
         100.00, 1);
 SET @Number = @Number + 1;
END;
GO

SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT ProductID, ProductName
FROM dbo.Products
WHERE ProductName = N'Practice product 1500';

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
```

Now run PDF 07 once to create the index, then run this identical query:

```
USE GreenBasketDB;
GO

-- First create the index using script 07.
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT ProductID, ProductName
FROM dbo.Products
WHERE ProductName = N'Practice product 1500';

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
```

Both versions must return ProductID 1500 and Practice product 1500. Repeat the SELECT measurement a few times under similar conditions. Do not clear shared server caches. Do not compare the loop insertion time with the query time.

## Performance record: fill after running

| Measure | Before index | After index |
|---|---|---|
| Returned rows | Expected: 1 | Expected: 1 |
| Plan operator | Record actual scan/seek | Record actual scan/seek |
| Logical reads | Record measured value | Record measured value |
| CPU / elapsed ms | Record measured values | Record measured values |

**Analysis:** A scan examines many rows; a seek uses the index to locate the target. The selective name query is expected to benefit, but the actual plan and reads decide. Small timings may be zero or affected by caching. If readings are similar, report that honestly. An index does not change the answer.

## Clean up the exercise data

```
USE GreenBasketDB;
GO

-- Run after saving the before/after evidence.
DELETE FROM dbo.Products
WHERE ProductID >= 1000 AND ProductID < 3000
 AND ProductName = N'Practice product '
  + CAST(ProductID AS NVARCHAR(10));
```

Keep the index for the portfolio. The cleanup removes only the reserved practice range with the expected generated names. The original three products remain.